AGENTIC AI TRAVEL PLANNER

Complete Product + Frontend + Backend + Agentic AI Master Blueprint

ONE DOCUMENT | NON-TECH EXPLANATION + TECHNICAL IMPLEMENTATION

Purpose: Is PDF ko founder, designer, non-technical stakeholder, engineer aur Codex - sab use kar sakte hain. Ye ek single source-of-truth specification hai, implemented-software claim nahi.

Product: ChatGPT-style travel assistant. User natural language mein multi-city trip batayega; system travel options research karega, realistic timetable aur costs compare karega, itinerary, maps, hotels, attractions aur booking redirects dikhayega.

Technology: React, TypeScript, Tailwind, FastAPI, Python, LangGraph, LangChain, RAG, PostgreSQL, pgvector, Redis, Docker aur AWS.

Non-technical stakeholder	Sections 1-4, 11, 14
Frontend engineer / designer	Sections 5-6, 10
Backend / AI engineer	Sections 7-13
Codex / coding assistant	Sections 15-16

Release objective: November 1, 2026 ka production-like V1 target; real-time integrations aur security hardening API access / testing par depend karte hain. Unverified prices kabhi “live” nahi dikhane.

1. Non-technical: Ye product kya karta hai?

Socho user bole: “Hyderabad se Pondicherry aur Rameshwaram jana hai, 3 din hain, budget ₹15,000 hai, aur Monday 10 AM se pehle office pahunchna hai.” User ko Google par 10 alag sites kholne ki zarurat kam honi chahiye.

- AI request ko samjhega: origin, locations, dates, traveler count, budget aur return deadline.
- Flight, train, bus, road routes aur hotels ke available options research karega.
- Attractions, ticket fees, opening hours aur tourist recommendations dikhaega.
- Day-wise travel schedule aur visual budget pie/bar charts dikhayega.
- Time/budget impossible ho to clear warning aur alternatives dega.
- Book karne ke liye trustworthy external booking pages open karega; booking itself provider par hogi.

What the system will NOT pretend to do

- Jahan live APIs nahi hain, wahaan “estimated / unknown” label use hoga; fake seats, fares, rooms ya crowd counts nahi.
- External booking redirect confirmed booking nahi hota. Payment aur personal passenger details third-party provider handle karega.
- Share Chat feature nahi hoga.

2. User journey - simple diagram

Visit website → Continue as Guest OR Sign In / Sign Up

↓

Select Temporary Chat OR Saved Chat (account required)

↓

Type travel request in conversational language

↓

AI researches, plans, validates and returns trip

↓

View route map, charts, hotels, places and booking links

↓

Ask follow-up: “Budget kam karo” → plan updates

3. Authentication and password recovery

Sign up	Name, email, password, confirmation.
Sign in	Email + password → secure session.
Guest access	Direct temporary exploration; no saved chat history.
Profile	Name, email, password change, logout and account deletion.
Forgot password	Email → 6-digit OTP → verify → new password. NO reset link.

OTP recovery flow

Enter email (generic response to avoid account enumeration)

↓

Generate secure 6-digit OTP and send via email provider

↓

Verify OTP (5 min expiry, max 5 attempts, 60 sec resend cooldown)

↓

Issue one-time short-lived reset authorization

↓

Enter and confirm new password → hash it → revoke older sessions

Engineering note: Store keyed OTP hashes, not raw OTP; password hash via a well-vetted algorithm. Secure HttpOnly cookies, CSRF controls and IP + account-level rate limits required.

4. Only two chat types

Who?	Guest or signed-in	Signed-in only
Sidebar?	Never	Yes
After tab close	Not recoverable from UI	Reopen until expiry
Storage	Tab memory + short-lived server processing	PostgreSQL with ownership controls
Expiry	Short TTL server cleanup	Fixed 7 days since creation
Sharing	No	No

Deletion nuance: A browser close event cannot guarantee instantaneous server deletion. Expired saved chats become inaccessible at read-time; cleanup removes messages, itinerary versions and agent checkpoints. Logs and backups require matching retention policy.

5. Frontend: screen map (7 agreed views)

1. Sign In	/login	Login, forgot password, guest entry
2. Sign Up	/signup	Registration and validation
3. Forgot/Reset	/forgot-password, /verify-otp, /reset-password	3-step email OTP workflow
4. Main Chat	/chat	New Chat, sidebar saved history, composer, profile menu
5. Trip Results	/chat/:id	Rich chat response: itinerary, map, hotels, transport, charts
6. Profile / Settings	/settings	Account and session controls
7. Temporary Chat	/chat?mode=temporary	Same chat UI with clear temporary-mode indicator

UI note: Temporary Chat is a mode, not a separate codebase. Trip Results are rich components embedded in the chat. Mobile sidebar collapses. Password recovery has multiple step routes but one logical screen group.

Frontend visual hierarchy

Chat Shell: Top bar + Left sidebar + Main conversation

↓

Conversation: User prompt → agent progress → AI response

↓

Rich Trip Result: Trip summary + tabs/cards

↓

Visuals: Route map + itinerary timeline + donut pie + comparison bars

↓

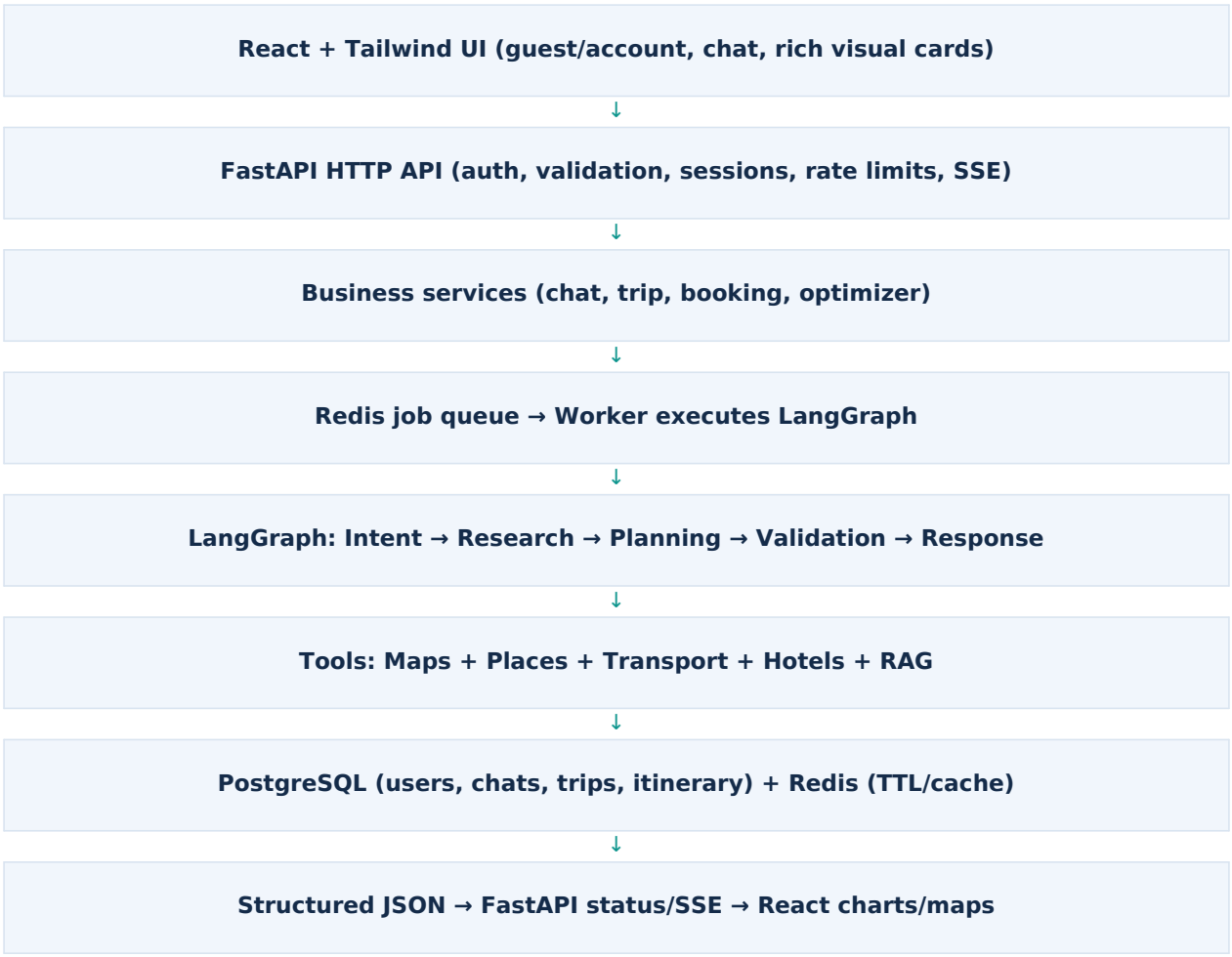
Actions: Modify plan, view booking provider, delete chat

6. Frontend folder structure and ownership

```
frontend/src/
  app/               App.tsx, router.tsx, providers.tsx
  pages/             LoginPage, SignupPage, ForgotPasswordPage,
                     VerifyOTPPage, ResetPasswordPage, ChatPage,
                     ProfilePage, NotFoundPage
  layouts/           AuthLayout, ChatLayout
  components/        ui/, sidebar/, common/
  features/          auth/, chat/, temporary-chat/, itinerary/,
                     transport/, hotels/, attractions/, maps/,
                     analytics/, profile/
  services/          apiClient.ts, authApi.ts, chatApi.ts, tripApi.ts
  hooks/             useAuth.ts, useChatStream.ts, useTrip.ts
  store/             authStore.ts, chatStore.ts
  types/             auth.ts, chat.ts, trip.ts
  utils/             dates.ts, currency.ts, formatters.ts
```

Charts: Recharts. Map: provider SDK. State: Zustand; server caching: TanStack Query. Frontend renders backend structured JSON; it must not invent prices, durations or availability.

7. Technical architecture: who talks to whom?



Principle: modular monolith

Backend ek main FastAPI repository hai, clear modules ke saath. Worker separately execute ho sakta hai, lekin har agent ke liye independent microservice nahi banani. Isse 3-day deadline mein integration easy rahega.

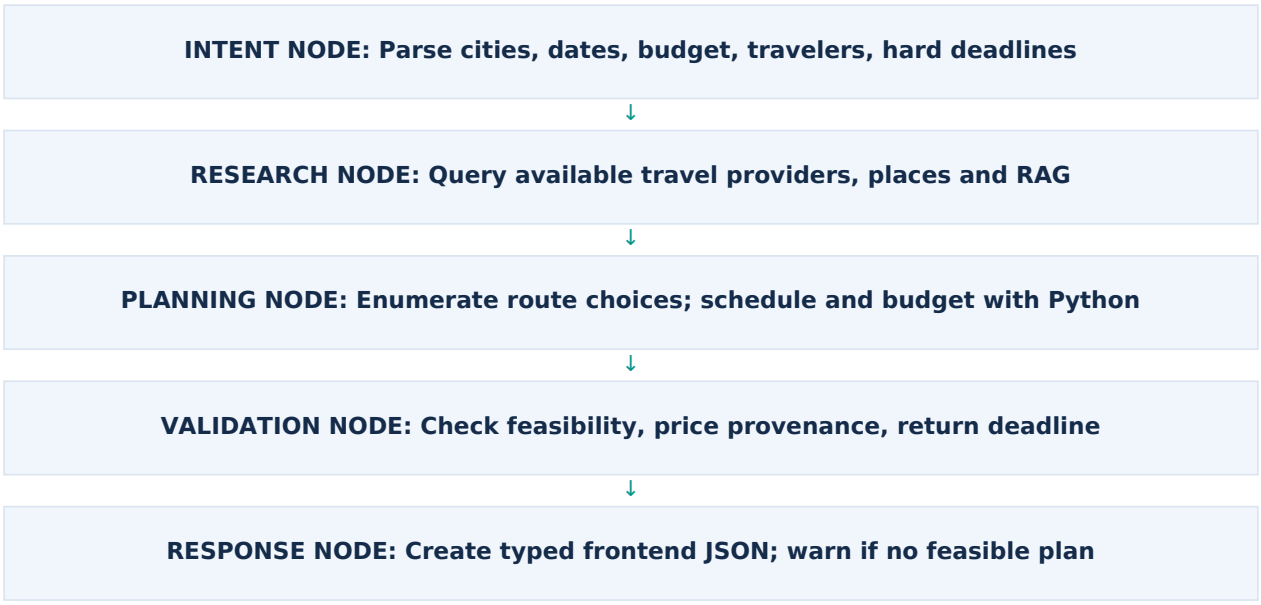
8. Backend folder structure - implementation map

```
backend/
  app/main.py
  app/api/v1/          auth.py, otp.py, users.py, conversations.py,
                      messages.py, temporary_chat.py, trips.py,
                      streaming.py, health.py
  app/core/            config.py, security.py, dependencies.py,
                      rate_limiter.py, logging.py, exceptions.py
  app/db/              base.py, session.py, checkpointer.py
  app/models/          user.py, auth_session.py, password_reset_otp.py,
                      conversation.py, message.py, trip.py,
                      itinerary.py, knowledge_document.py
  app/schemas/         auth.py, otp.py, user.py, conversation.py,
                      message.py, trip.py, itinerary.py,
                      transport.py, hotel.py, attraction.py, budget.py
  app/repositories/    user_repository.py, session_repository.py,
                      otp_repository.py, conversation_repository.py,
                      message_repository.py, trip_repository.py,
                      itinerary_repository.py
  app/services/        auth_service.py, otp_service.py, email_service.py,
                      conversation_service.py, temporary_chat_service.py,
                      chat_service.py, trip_service.py, route_service.py,
                      transport_service.py, hotel_service.py,
                      attraction_service.py, budget_service.py,
                      itinerary_service.py, booking_service.py,
                      cleanup_service.py
  app/agents/          graph.py, state.py, llm.py, prompts.py,
                      edges.py, nodes/{intent,research,planning,
                      validation,response}_node.py
  app/integrations/    base.py, maps_provider.py, places_provider.py,
                      flight_provider.py, train_provider.py,
                      bus_provider.py, hotel_provider.py,
                      booking_provider.py
  app/rag/             ingestion.py, embeddings.py, vector_store.py,
                      retriever.py, documents/
  app/tasks/           celery_app.py, trip_tasks.py, cleanup_tasks.py
  app/utils/           datetime_utils.py, currency_utils.py,
                      url_validator.py
  alembic/; tests/; scripts/; Dockerfile; pyproject.toml;
  .env.example; README.md
```

9. Backend: file-by-file responsibilities

main.py, api/router	Create FastAPI app, CORS/middleware, register versioned routes.
api/v1/auth.py, otp.py	Registration/login/logout and OTP send/verify/resend/reset endpoints.
api/v1/conversations.py, messages.py	Owned saved-chat CRUD; append messages; check 7-day expiry.
api/v1/temporary_chat.py, streaming.py	No-history temporary prompt; SSE progress; job status.
core/config.py, security.py, dependencies.py	Typed env settings, password/session helpers, auth dependency.
core/rate_limiter.py, logging.py	Anti-abuse budgets, log IDs and redaction.
db/base.py, session.py, checkpointer.py	SQLAlchemy tables/session + short-retention LangGraph persistence.
models/*.py	Users, auth sessions, OTP, chats, messages, trips, itineraries, knowledge.
schemas/*.py	Pydantic request/response contracts including budget/chart types.
repositories/*.py	Safe database queries: ownership filtering, save, list, delete.
services/auth*, otp*, email*	Credentials, OTP lifecycle, email provider adapter.
services/chat*, conversation*	Chat mode routing, turns, history and expiry.
services/trip*, itinerary*, budget*	Trip workflow execution, calculation, validations and versions.
services/route*, transport*, hotel*, attraction*	Normalize provider options, travel scheduling and pricing sources.
agents/graph.py, state.py, nodes/*	Graph edges, shared typed state, 5 nodes and retries.
integrations/*_provider.py	Isolate live provider APIs, timeouts, caching and unavailable states.
rag/*	Curated travel documents ingestion and vector retrieval.
tasks/*, cleanup_service.py	Asynchronous trip jobs; cleanup expired chats/checkpoints.
tests/*	Auth, OTP, sessions, expiry, planning and API security tests.

10. LangGraph: how AI actually plans a trip



Conditional flow: Missing critical input → ask question. Invalid dates → clarification. Failed feasibility → retry with bounded alternative generation. Still impossible → explain why with safer choices.

Required graph state fields

```
TravelState:
  conversation_id, user_prompt, requirements, previous_trip
  route_options, transport_options, hotels, attractions
  destination_knowledge, itinerary, budget_breakdown
  validation_errors, warnings, retry_count, status
  provider_sources, fetched_at, final_response
```

Who does calculation?

LLM: Understand intent, ask clarifications, summarize choices, format explanations.
Python tools: Date arithmetic, budget totals, deadline checks, route comparison, currency normalization.
Providers: Live prices, availability, distance, routes, locations if APIs allow.
RAG: Evergreen destination guidance, not live seats or fares.

11. Example user input and result

Example input: “30 Oct to 1 Nov, Hyderabad → Pondicherry → Rameshwaram → Hyderabad, office by 2 Nov 10 AM, ₹15,000.”

Intent	Origin Hyderabad, two destinations, dates, budget, deadline.
Research	Transport + hotel options with sources or unknown states.
Plan	Candidate orders, travel legs, day-wise activities and cost buckets.
Validate	Reject impossible connections and deadline misses.
Display	Day timeline, route map, cost donut, comparison bars, booking URLs.

Important: The example is illustrative. A feasible itinerary must be based on route and schedule evidence; never treat illustration as a confirmed offer.

12. Database design and data lifecycle

users	Account identity and password hash.
auth_sessions	Secure sessions, expiry and revocation.
password_reset_otps	OTP hash, attempts, expiry and reset authorization.
conversations	Owner, title, created_at and fixed expires_at.
messages	Conversation ID, role, content, timestamp.
trips	Current normalized trip request and generation status.
itineraries	Versioned plan JSON linked to chat/trip.
knowledge_documents	Curated RAG metadata and embedding (pgvector).

Retention and privacy

- Temporary conversation must never appear in saved history; browser state is tab scoped; backend execution state short lived.
- Saved chat expires at creation time + 7 days (not extended on new messages); API filters expired records.
- Scheduled cleanup must delete linked messages, itinerary versions and graph checkpoints; account deletion follows same linkage.
- Keep secret provider keys server-side; minimize personal data and scrub sensitive content from logs.

13. REST API contract - build these first

POST	/api/v1/auth/signup	Create account
POST	/api/v1/auth/login	Sign in + session cookie
POST	/api/v1/auth/logout	Revoke session
POST	/api/v1/auth/forgot-password	Email OTP request
POST	/api/v1/auth/verify-otp	Verify OTP
POST	/api/v1/auth/resend-otp	Resend OTP
POST	/api/v1/auth/reset-password	Save new password
GET/PATCH	/api/v1/users/me	Profile
GET/POST	/api/v1/conversations	List/create saved chats
GET/DELETE	/api/v1/conversations/{id}	Read/delete owned chat
POST	/api/v1/conversations/{id}/messages	Saved chat turn
POST	/api/v1/chat/temporary	Temporary chat turn
POST	/api/v1/trips/plan	Start trip generation
GET	/api/v1/trips/{id}	Fetch structured plan
GET	/api/v1/trips/{id}/events	Stream planning progress
GET	/api/v1/health + /ready	Liveness/readiness

14. Structured outputs for frontend graphs

Backend sends typed JSON; frontend only renders. Every fare quote is tagged live_quote / estimate / unknown with source and fetched_at.

```
{
  "trip_id": "trip_demo",
  "status": "completed",
  "feasible": true,
  "warnings": [],
  "itinerary": [{"day": 1, "activities": []}],
  "routes": [{"origin":"Hyderabad","destination":"Pondicherry"}],
  "transport_options": [],
  "hotel_options": [],
  "attractions": [],
  "budget": {
    "currency":"INR", "total_estimated":15000,
    "price_type":"estimate",
    "categories":[
      {"name":"Transport","amount":6000},
      {"name":"Hotels","amount":4000},
      {"name":"Food","amount":2000},
      {"name":"Attractions","amount":1500},
      {"name":"Buffer","amount":1500}
    ]
  },
  "booking_options": []
}
```

Data sample only: These ₹ values are dummy chart examples and must not be presented as verified fares.

Travel provider architecture

Research Node → services → provider adapter → external API. Train/flight/bus/hotel provider wrappers normalize their different response structures. All network calls have timeouts, retries, validation, caching and source labels.

UX charts to implement

- Budget doughnut/pie chart with category totals.
- Transport price and duration comparison (bar chart + cards).
- Day-wise spending graph and budget progress indicator.
- Interactive route map and day-wise activity timeline.

15. Security, reliability and cloud deployment

Application	Input bounds, Pydantic validation, session ownership.
Authentication	Secure cookies, CSRF defense, strong hashes, OTP rate limits.
Public AI	IP/session quotas, cost caps, concurrency limits, bounded graph retries.
LLM tools	No arbitrary shell/network access; provider allowlists; injection defense.
Database	Migrations, least-privilege DB roles, backup retention.
Provider calls	Retries, timeouts, source timestamps, no fake availability.
Monitoring	Structured request logs, error tracking, job metrics, alerts.
Cloud	HTTPS, secrets manager, WAF, restricted networking.

Docker / AWS target



Release caveat: November 1 target is production-like V1, not guaranteed enterprise production readiness. Provider access, testing and deployment may force transparent scope tradeoffs.

16. Codex implementation instructions and order

Codex master prompt: “Read this entire PDF as the architecture source of truth. Inspect existing files first. Build a modular FastAPI backend and feature-based React frontend matching these contracts. Implement one module per iteration. Before coding, list proposed changed files. After coding, explain every file/function in simple Hinglish; run tests, report results, and stop. Do not fabricate live travel data, weaken security or silently change the retention policy.”

1	Backend foundation / Docker / schemas	/health and /ready work
2	Minimal graph → trip JSON	One sample prompt produces validated JSON
3	Auth + email OTP	Signup/login/reset end-to-end passes
4	Temporary + saved chats	7-day expiry and ownership tested
5	Maps + places + provider adapters + RAG	Sources / unknown states displayed correctly
6	Frontend + charts + streaming	Complete user journey runs
7	Deploy + secure + regression test	HTTPS, errors, quotas, cleanup verified

17. Acceptance checklist - non-tech and tech teams

- Guest user can open a temporary conversation without signing up.
- Registered user can sign in, see saved history and continue a non-expired chat.
- Email OTP → verification → new password works; no reset email links.
- Saved chats expire seven days after creation and cannot be accessed afterward.
- Temporary chats do not appear in saved history; closing tab prevents UI recovery.
- Natural language allows multiple cities in any order plus date/budget constraints.
- AI generates feasible or explicitly infeasible itinerary with alternatives.
- Results include maps, day timeline, hotel/place cards, budget charts and booking redirects.
- Quotes reveal source, timestamp and estimate/live/unknown provenance.
- Basic security, failures, timeouts and rate limits are tested.
- Deployment works over HTTPS and critical logs are captured.

18. Common words explained (for non-technical reviewers)

Frontend	What people see and click in the browser.
Backend	Server logic that processes requests.
API	Structured way two systems exchange data.
Agent / LangGraph	Workflow deciding what AI/tool step runs next.
RAG	AI looks up relevant curated documents before answering.
Redis	Fast temporary storage / queue.
PostgreSQL	Durable application database.
Vector search	Search documents by semantic similarity.
SSE	Streams progress from backend to frontend.
Docker / AWS	Packaging and hosting of the application.

FINAL PRODUCT VISION: ChatGPT-style interaction + reliable multi-city travel planning + beautiful, source-aware itinerary analytics.